```
const accessKey = 'wIV1ZG1i7YTr7bl4RLumfNkd2FZzM2L69zzww0GULSM';  // Твой API-ключ
document.getElementById('showBtn').addEventListener('click', async () => {  // Слушатель клика на кнопку
  const query = document.getElementById('searchQuery').value.trim();  // Получаем ключевое слово из инпута, убираем пробелы
  const count = parseInt(document.getElementById('imageCount').value);  // Получаем число изображений, преобразуем в int
  if (!query || isNaN(count) || count < 2) {  // Валидация: query не пустой, count — число >=2
    alert('Keyword cannot be empty and number of images must be at least 2.');  // Алерт с ошибкой
    return;  // Выходим из функции}
  const resultsDiv = document.getElementById('gallery');  // Находим див для галереи
  resultsDiv.innerHTML = '';  // Очищаем предыдущие результаты
  const url = `https://api.unsplash.com/search/photos?query=${encodeURIComponent(query)}&per_page=${count}&client_id=${accessKey}`;  // Формируем URL с параметрами (encodeURIComponent для безопасного кодирования query)
  console.log('Fetching URL:', url);  // Лог для отладки
  try {  // Пробуем выполнить запрос
    const response = await fetch(url);  // Асинхронный запрос с fetch
    if (!response.ok) {  // Проверяем статус (200-299 = ok)
      throw new Error(`API request failed with status ${response.status}`);  }
    const data = await response.json();  // Парсим JSON из ответа
    data.results.forEach(photo => {  // Для каждого фото в результатах
      const orientation = photo.width > photo.height ? 'landscape' :  // Определяем ориента
                  photo.height > photo.width ? 'portrait' : 'square';
      const card = document.createElement('div');  // Создаём карточку
      card.className = 'image-item';  // Класс для стилей
      card.innerHTML = `  // HTML для карточки (изображение + детали)
        <img src="${photo.urls.small}" alt="${photo.alt_description || photo.description || 'Image'}">
        <div class="image-details">
          <p><strong>Photographer:</strong> ${photo.user.name}</p>
          <p><strong>Description:</strong> ${photo.description || photo.alt_description || 'None'}</p>
          <p><strong>Likes:</strong> ${photo.likes}</p>
          <p><strong>Image URL:</strong> <a href="${photo.links.html}" target="_blank">${photo.links.html}</a></p>
          <p><strong>Orientation:</strong> ${orientation}</p>
          <p><strong>Full URL:</strong> ${photo.urls.full}</p>
        </div> `;
      resultsDiv.appendChild(card);  // Добавляем карточку в галерею
    });
  } catch (error) {  // Ловим ошибки
    console.error(error);  // Лог в консоль
    alert('Error fetching images: ' + error.message);  // Алерт пользовател }
});
AXIOS const accessKey = 'wIV1ZG1i7YTr7bl4RLumfNkd2FZzM2L69zzww0GULSM';  // Твой API-ключ
document.getElementById('showBtn').addEventListener('click', async () => {  // Слушатель клика
 const query = document.getElementById('searchQuery').value.trim();  // Ключевое слово
 const count = parseInt(document.getElementById('imageCount').value, 10);  // Число (с базой 10 для parseInt)
 if (!query || isNaN(count) || count < 2) {  // Валидация
  alert('Keyword cannot be empty and number of images must be at least 2.');
  return; }
 const resultsDiv = document.getElementById('gallery');  // Галерея
 resultsDiv.innerHTML = '';  // Очистка
 try {  // Пробуем запрос
  const res = await axios.get('https://api.unsplash.com/search/photos', {
    params: {
     query: query,
     per_page: count,
     client_id: accessKey   } });
  const data = res.data;  // Данные из ответа (axios автоматически парсит JSON)
  if (!data || !data.results || data.results.length === 0) {  // Проверка на пустые результаты
   alert('No results found.');  return; }
  data.results.forEach(photo => {  // Для каждого фото
   const orientation = photo.width > photo.height ? 'landscape' :  // Ориентация
               photo.height > photo.width ? 'portrait' : 'square';
   const card = document.createElement('div');  // Карточка
   card.className = 'image-item';
   card.innerHTML = `  // HTML (похожий, но с ссылкой на full URL и optional chaining для user.name)
EJJJJJJJJJJS <!doctype html>
<html lang="en">
<head>
 <meta charset="utf-8"/>
  <meta name="viewport" content="width=device-width,initial-scale=1"/>
 <title>Unsplash Gallery (Server-rendered)</title>
 <style>
    body{ font-family: Inter, system-ui, sans-serif; background:#f8fafc; margin:20px; color:#0f172a; }
  .container{ max-width:1100px; margin:0 auto; }
  .controls{ display:flex; gap:8px; align-items:center; margin-bottom:12px; }
        input[type="text"], input[type="number"]{ padding:8px; border-radius:8px; border:1px solid #e6eef6; }
     button{ padding:8px 12px; border-radius:8px; border:0; background:#0ea5e9; color:white; cursor:pointer; }
  .error{ color:#b91c1c; margin-bottom:12px; }
        .gallery{ display:grid; grid-template-columns:repeat(auto-fill,minmax(260px,1fr)); gap:16px; }
     .card{ background:white; border-radius:12px; overflow:hidden; box-shadow:0 10px 30px rgba(2,6,23,0.06); }
    .card img{ width:100%; height:180px; object-fit:cover; display:block; }
 </style>
 <script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
</head>
<body>
 <div class="container">
    <h1>Unsplash Gallery — Server (Express + EJS)</h1>

  <% if(error){ %>
   <div class="error"><%= error %></div>
  <% } %>
  <form action="/search" method="post" novalidate>
   <div class="controls">
        <input type="text" name="searchQuery" placeholder="Search keyword" value="<%= typeof query !== 'undefined' ? query : '' %>" />
       <input type="number" name="imageCount" min="1" value="<%= typeof count !== 'undefined' ? count : 4 %>" />
    <button type="submit">Search (server)</button>
   </div>
  </form>
  <hr/>
  <!-- Client-side search (AJAX) -->
  <div style="margin:12px 0;">
        <input id="searchQuery" type="text" placeholder="Client search keyword (AJAX)"/>
    <input id="imageCount" type="number" min="1" value="4"/>
        <button id="showBtn">Show Images (client)</button>
  </div>
  <!-- Server side search -->
  <div id="gallery" class="gallery">
   <% images.forEach(function(item){ %>
    <div class="image-item">
      <img src="<%= item.urls.small %>" alt="<%= item.description || item.alt_description || 'Image' %>">
```

```
    <img src="${photo.urls.small}" alt="${photo.alt_description || photo.description || 'Image'}">
    <div class="image-details">
     <p><strong>Photographer:</strong> ${photo.user?.name || 'Unknown'}</p>  // ?. — безопасный доступ, если user undefined
     <p><strong>Description:</strong> ${photo.description || photo.alt_description || 'None'}</p>
     <p><strong>Likes:</strong> ${photo.likes}</p>
     <p><strong>Image URL:</strong> <a href="${photo.links.html}" target="_blank" rel="noopener">${photo.links.html}</a></p>
     <p><strong>Orientation:</strong> ${orientation}</p>
     <p><strong>Full URL:</strong> <a href="${photo.urls.full}" target="_blank" rel="noopener">View full</a></p>
    </div>  `;
   resultsDiv.appendChild(card);  });
 } catch (err) {  // Ошибки
  console.error('Axios error:', err);  // Лог
  const message = err.response ? `API error ${err.response.status}` : err.message;  // Более детальная ошибка (axios даёт err.response)
  alert('Error fetching images: ' + message);  // Алерт  }});
```

```
    <div class="image-details">
          <p><strong>Photographer:</strong> <%= item.photographer %></p>
          <p><strong>Description:</strong> <%= item.description || 'None' %></p>
          <p><strong>Likes:</strong> <%= item.likes %></p>
        <p><strong>Image URL:</strong>
              <a href="<%= item.links.html %>" target="_blank"><%= item.links.html %></a>
        </p>
              <p><strong>Orientation:</strong> <%= item.orientation %></p>
        <p><strong>Full URL:</strong>
         <a href="<%= item.urls.full %>" target="_blank" rel="noopener">View full</a></p>  </div>  </div>
    <% }); %>
  </div> </div>
 <!-- client-side scripts from public/ -->
 <!-- <script src="/fetchapi.js"></script> -->
 <!-- <script src="/axios.js"></script> </body></html>
```

SERVER.JS

```
import express from 'express';
import axios from 'axios';
import dotenv from 'dotenv';
import path from 'path';
import { fileURLToPath } from 'url';

dotenv.config();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const port = process.env.PORT || 3000;
const accessKey = process.env.UNSPLASH_ACCESS_KEY; // from .env

if (!accessKey) {
 console.warn('WARNING: No UNSPLASH_ACCESS_KEY found in .env');
}

// view engine + static + body parsing
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// GET / -> render empty page but pass all template vars
app.get('/', (req, res) => {
 res.render('unsplashGallery', {
  images: [],
  error: null,
  query: '',
  count: 4
 });
});// POST /search -> validate, fetch from Unsplash, map results, render
app.post('/search', async (req, res) => {
 try {
  const query = (req.body.searchQuery  || '').trim();
  const count = parseInt(req.body.imageCount, 10) || 0;

  // Validation
  if (!query) {
   return res.status(400).render('unsplashGallery', {
    images: [],
    error: 'Search keyword is required.',
    query,
    count
   })  }
  if (Number.isNaN(count) || count < 2) {
   return res.status(400).render('unsplashGallery', {
    images: [],
    error: 'Number of images must be at least 2.',
    query,
    count
   });
  }
 consturl= `https://api.unsplash.com/search/photos?query=${encodeURIComponent(query)}&per_page=${count}&client_id=${accessKey}`;

 // //Axios GET request
  // const response = await axios.get(url);
  // const results = response.data.results || [];

 // Fetch API GET request
  const response = await fetch(url);
  const results = (await response.json()).results || [];

  // Map Unsplash items to the shape EJS
  const images = results.map(item => ({
   photographer: item.user && item.user.name ? item.user.name : 'Unknown',
   description: item.alt_description || item.description || '',
   likes: item.likes || 0,
   urls: item.urls || {},
   links: item.links || {},
   width: item.width,
   height: item.height,
      orientation: (item.width && item.height) ? (item.width > item.height ? 'landscape' : (item.width < item.height ? 'portrait' : 'square')) : 'unknown'
  }));

  res.render('unsplashGallery', { images, error: null, query, count });
 } catch (err) {
  console.error('Server error:', err?.response?.data || err.message || err);
  res.status(500).render('unsplashGallery', {
   images: [],
   error: 'Server error: failed to fetch images.',
   query: req.body.query || '',
   count: req.body.count || 4
  });
 }
});

app.listen(port, () => {
 console.log(`Server running on http://localhost:${port}`);
});
```